

SPECIALIZED_PLATFORMS_DEPLOYMENT

Specialized Platforms Deployment Guide

Overview

This guide covers deployment of HelixCode on specialized platforms: Aurora OS (Russian Federation) and Harmony OS (China/Global). These platforms offer unique features and require specific deployment considerations.

Table of Contents

1. [Prerequisites](#)
 2. [Deployment Methods](#)
 3. [Aurora OS Deployment](#)
 4. [Harmony OS Deployment](#)
 5. [Combined Deployment](#)
 6. [Production Configuration](#)
 7. [High Availability Setup](#)
 8. [Monitoring and Maintenance](#)
 9. [Security Hardening](#)
 10. [Troubleshooting](#)
-

Prerequisites

System Requirements

Aurora OS

- **OS Version:** Aurora OS 4.0 or later
- **Architecture:** x86_64 or aarch64
- **RAM:** 4 GB minimum (8 GB recommended)
- **Storage:** 500 MB free space
- **Database:** PostgreSQL 13+
- **Optional:** Redis 6.0+ for caching

Harmony OS

- **OS Version:** Harmony OS 3.0+ (4.0+ recommended)
- **Architecture:** aarch64 (Kirin chipsets)
- **RAM:** 4 GB minimum (8 GB recommended for distributed computing)
- **Storage:** 1 GB free space
- **Database:** PostgreSQL 13+
- **Redis:** Recommended for distributed features
- **NPU:** Kirin 990+ for AI acceleration (optional)

Software Requirements

```
# PostgreSQL
```

```
postgresql >= 13.0
```

```
# Redis (optional but recommended for Harmony OS)
```

```
redis >= 6.0
```

```
# System utilities
```

```
systemd (for service management)
```

```
openssl (for certificate generation)
```

```
curl (for health checks)
```

Network Requirements

Aurora OS

- Port 8080: HTTP API (configurable)
- Port 5432: PostgreSQL (if external)
- Port 6379: Redis (if external)

Harmony OS

- Port 8080: HTTP API (configurable)
 - Port 8081: Peer discovery
 - Port 8082: Data transfer
 - Port 5432: PostgreSQL (if external)
 - Port 6379: Redis (if external)
-

Deployment Methods

Method 1: Automated Deployment (Recommended)

Using the provided deployment scripts for quick, production-ready deployments.

Pros: - Fast and automated - Consistent configuration - Production-ready defaults - Service management included

Cons: - Less control over details - May require customization post-deployment

Method 2: Manual Deployment

Step-by-step manual installation for custom requirements.

Pros: - Full control - Custom configurations - Better understanding of components

Cons: - More time-consuming - Higher chance of misconfiguration

Method 3: Containerized Deployment

Using Docker or similar container technology (future implementation).

Pros: - Isolated environment - Reproducible deployments - Easy scaling

Cons: - Requires container runtime - Additional complexity

Aurora OS Deployment

Automated Deployment

Step 1: Build Binary

```
cd HelixCode  
make aurora-os
```

Verify the binary:

```
ls -lh bin/aurora-os  
file bin/aurora-os
```

Step 2: Run Deployment Script

```
sudo ./scripts/deploy-aurora-os.sh
```

The script performs: 1. Root privilege verification 2. Aurora OS detection 3. Binary verification 4. Dependency checks 5. User and directory creation 6. Binary installation 7. Configuration file creation 8. Systemd service setup 9. Permission configuration

Step 3: Configure Environment

Edit /etc/helixcode/aurora.env:

```
# Database credentials
HELIX_DATABASE_PASSWORD=your_secure_password_here

# JWT secret for authentication
HELIX_AUTH_JWT_SECRET=$(openssl rand -hex 32)

# Optional: Redis password
HELIX_REDIS_PASSWORD=your_redis_password

# Aurora OS specific
AURORA_SECURITY_LEVEL=enhanced
AURORA_MONITORING_ENABLED=true
```

Step 4: Database Setup

```
# Create database and user
sudo -u postgres createdb helixcode
sudo -u postgres createuser helixcode

# Set password
sudo -u postgres psql <<EOF
ALTER USER helixcode WITH PASSWORD 'your_secure_password_here';
GRANT ALL PRIVILEGES ON DATABASE helixcode TO helixcode;
EOF

# Test connection
psql -h localhost -U helixcode -d helixcode -c "SELECT
    version();"

```

Step 5: Start Service

```
# Start the service
sudo systemctl start helixcode-aurora

# Enable auto-start on boot
sudo systemctl enable helixcode-aurora
```

```
# Check status
sudo systemctl status helixcode-aurora
```

```
# View logs
sudo journalctl -u helixcode-aurora -f
```

Step 6: Verify Deployment

```
# Health check
curl http://localhost:8080/health
```

```
# API test
curl http://localhost:8080/api/version
```

```
# Check process
ps aux | grep aurora-os
```

```
# Check listening ports
sudo ss -tlnp | grep 8080
```

Manual Deployment

Step 1: Create User and Directories

```
# Create system user
sudo useradd -r -s /bin/false -d /var/lib/helixcode helixcode
```

```
# Create directories
sudo mkdir -p /opt/helixcode
sudo mkdir -p /etc/helixcode
sudo mkdir -p /var/log/helixcode
sudo mkdir -p /var/lib/helixcode
```

```
# Set ownership
sudo chown -R helixcode:helixcode /opt/helixcode
sudo chown -R helixcode:helixcode /etc/helixcode
sudo chown -R helixcode:helixcode /var/log/helixcode
sudo chown -R helixcode:helixcode /var/lib/helixcode
```

Step 2: Install Binary

```
# Copy binary
sudo cp bin/aurora-os /opt/helixcode/
```

```
sudo chmod +x /opt/helixcode/aurora-os
```

```
# Create symlink
```

```
sudo ln -s /opt/helixcode/aurora-os /usr/local/bin/helixcode-aurora
```

```
# Verify
```

```
/usr/local/bin/helixcode-aurora --version
```

Step 3: Create Configuration

```
sudo tee /etc/helixcode/aurora-config.yaml > /dev/null <<EOF
```

```
server:
```

```
  address: "0.0.0.0"
  port: 8080
  tls_enabled: false
```

```
database:
```

```
  host: "localhost"
  port: 5432
  user: "helixcode"
  dbname: "helixcode"
  sslmode: "prefer"
```

```
redis:
```

```
  enabled: false
  host: "localhost"
  port: 6379
```

```
auth:
```

```
  token_expiry: 86400
  session_expiry: 604800
```

```
aurora:
```

```
  enable_security_features: true
  enable_system_monitoring: true
  enable_native_integration: true
  security_level: "enhanced"
```

```
workers:
```

```
  health_check_interval: 30
  max_concurrent_tasks: 10
```

```
logging:
```

```
    level: "info"
    file: "/var/log/helixcode/aurora-os.log"
EOF
```

```
# Set permissions
sudo chmod 644 /etc/helixcode/aurora-config.yaml
```

Step 4: Create Systemd Service

```
sudo tee /etc/systemd/system/helixcode-aurora.service > /dev/
    null <<EOF
[Unit]
Description=HelixCode Aurora OS Client
After=network.target postgresql.service
Wants=postgresql.service

[Service]
Type=simple
User=helixcode
Group=helixcode
WorkingDirectory=/opt/helixcode
ExecStart=/opt/helixcode/aurora-os
Restart=always
RestartSec=10
StandardOutput=journal
StandardError=journal

# Environment
Environment="HELIX_CONFIG_PATH=/etc/helixcode/aurora-config.yaml"
EnvironmentFile=-/etc/helixcode/aurora.env

# Security hardening
NoNewPrivileges=true
PrivateTmp=true
ProtectSystem=strict
ProtectHome=true
ReadWritePaths=/var/log/helixcode /var/lib/helixcode

[Install]
WantedBy=multi-user.target
EOF

# Reload systemd
sudo systemctl daemon-reload
```

Harmony OS Deployment

Automated Deployment

Step 1: Build Binary

```
cd HelixCode
make harmony-os
```

Verify:

```
ls -lh bin/harmony-os
file bin/harmony-os
```

Step 2: Run Deployment Script

```
sudo ./scripts/deploy-harmony-os.sh
```

The script performs: 1. Root privilege verification 2. Harmony OS detection (checks for hdc, harmonyos-release) 3. Binary verification 4. Dependency checks (systemd or init.d) 5. User and directory creation (including distributed directories) 6. Binary installation 7. Configuration file creation 8. Service setup (systemd or init.d) 9. Permission configuration

Step 3: Configure Environment

Edit /etc/helixcode/harmony.env:

```
# Database credentials
HELIX_DATABASE_PASSWORD=your_secure_password_here

# Redis credentials (required for distributed features)
HELIX_REDIS_PASSWORD=your_redis_password_here

# JWT secret
HELIX_AUTH_JWT_SECRET=$(openssl rand -hex 32)

# Harmony OS specific
HARMONY_DEVICE_ID=auto
HARMONY_SUPER_DEVICE_ENABLED=true
HARMONY_NPU_ENABLED=true
HARMONY_GPU_ENABLED=true
HARMONY_DISTRIBUTED_ENABLED=true
```


Step 4: Database and Redis Setup

PostgreSQL

```
sudo -u postgres createdb helixcode
sudo -u postgres createuser helixcode
sudo -u postgres psql <<EOF
ALTER USER helixcode WITH PASSWORD 'your_secure_password_here';
GRANT ALL PRIVILEGES ON DATABASE helixcode TO helixcode;
EOF
```

Redis (if not already installed)

```
sudo apt install redis-server
sudo systemctl enable redis-server
sudo systemctl start redis-server
```

Configure Redis password

```
sudo redis-cli CONFIG SET requirepass "your_redis_password_here"
sudo redis-cli CONFIG REWRITE
```

Test Redis

```
redis-cli -a "your_redis_password_here" PING
```

Step 5: Start Service

For systemd

```
sudo systemctl start helixcode-harmony
sudo systemctl enable helixcode-harmony
sudo systemctl status helixcode-harmony
```

For init.d

```
sudo /etc/init.d/helixcode-harmony start
sudo /etc/init.d/helixcode-harmony status
```

View logs

```
sudo journalctl -u helixcode-harmony -f
```

or

```
sudo tail -f /var/log/helixcode/harmony-os.log
```

Step 6: Verify Deployment

Health check

```
curl http://localhost:8080/health
```

Distributed engine status

```
helixcode-harmony distributed status
```

```
# Device status (NPU/GPU)
```

```
helixcode-harmony device list
```

```
# Check process
```

```
ps aux | grep harmony-os
```

Manual Deployment

Similar to Aurora OS manual deployment, with these additions:

Additional Directories

```
sudo mkdir -p /var/lib/helixcode/distributed
```

```
sudo mkdir -p /var/lib/helixcode/services
```

```
sudo chown -R helixcode:helixcode /var/lib/helixcode
```

Harmony OS Configuration

```
harmony:
```

```
# Distributed computing
```

```
enable_distributed_computing: true
```

```
enable_cross_device_sync: true
```

```
sync_interval: 30
```

```
# Resource management
```

```
enable_resource_optimization: true
```

```
enable_ai_acceleration: true
```

```
gpu_enabled: true
```

```
npu_enabled: true
```

```
# System integration
```

```
enable_system_monitoring: true
```

```
enable_multi_screen: true
```

```
enable_super_device: true
```

```
# Service coordination
```

```
service_discovery_enabled: true
```

```
service_failover_enabled: true
```

```
health_check_interval: 15
```

Combined Deployment

Deploy Both Platforms

Use the combined deployment script:

```
# Build both
make aurora-harmony

# Deploy both (interactive mode)
sudo ./scripts/deploy-specialized-platforms.sh

# Or specify both explicitly
sudo ./scripts/deploy-specialized-platforms.sh --platform both

# With build and clean options
sudo ./scripts/deploy-specialized-platforms.sh \
  --platform both \
  --build \
  --clean
```

Port Configuration for Co-Location

When running both on the same server:

Aurora OS (/etc/helixcode/aurora-config.yaml):

```
server:
  port: 8080
```

Harmony OS (/etc/helixcode/harmony-config.yaml):

```
server:
  port: 8081

harmony:
  distributed:
    peer_discovery_port: 8091
    data_transfer_port: 8092
```

Managing Both Services

```
# Start both
sudo systemctl start helixcode-aurora helixcode-harmony
```

```
# Stop both
sudo systemctl stop helixcode-aurora helixcode-harmony

# Restart both
sudo systemctl restart helixcode-aurora helixcode-harmony

# Enable both on boot
sudo systemctl enable helixcode-aurora helixcode-harmony

# Status of both
sudo systemctl status helixcode-aurora helixcode-harmony
```

Production Configuration

TLS/SSL Configuration

Generate Certificates

```
# Self-signed certificate (for testing)
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \
  -keyout /etc/helixcode/key.pem \
  -out /etc/helixcode/cert.pem \
  -subj "/CN=helixcode.local"

# Set permissions
sudo chmod 600 /etc/helixcode/key.pem
sudo chmod 644 /etc/helixcode/cert.pem
sudo chown helixcode:helixcode /etc/helixcode/*.pem
```

Enable TLS

```
server:
  tls_enabled: true
  tls_cert: "/etc/helixcode/cert.pem"
  tls_key: "/etc/helixcode/key.pem"
```

Database Optimization

PostgreSQL Configuration

```
-- Connection pooling
ALTER SYSTEM SET max_connections = 200;
ALTER SYSTEM SET shared_buffers = '256MB';
```

```
ALTER SYSTEM SET effective_cache_size = '1GB';
ALTER SYSTEM SET maintenance_work_mem = '64MB';
ALTER SYSTEM SET checkpoint_completion_target = 0.9;
ALTER SYSTEM SET wal_buffers = '16MB';
ALTER SYSTEM SET default_statistics_target = 100;
ALTER SYSTEM SET random_page_cost = 1.1;
ALTER SYSTEM SET effective_io_concurrency = 200;
```

```
-- Reload configuration
SELECT pg_reload_conf();
```

Connection Pooling

```
database:
  max_open_conns: 100
  max_idle_conns: 10
  conn_max_lifetime: 3600
```

Redis Configuration

```
redis:
  enabled: true
  max_retries: 3
  pool_size: 10
  min_idle_conns: 5
  max_conn_age: 3600
```

Log Rotation

Create /etc/logrotate.d/helixcode:

```
/var/log/helixcode/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 0644 helixcode helixcode
    sharedscripts
    postrotate
        systemctl reload helixcode-aurora 2>/dev/null || true
        systemctl reload helixcode-harmony 2>/dev/null || true
    endscript
}
```

High Availability Setup

Load Balancer Configuration

HAProxy Example

```
frontend helixcode_front
    bind *:80
    default_backend helixcode_backend

backend helixcode_backend
    balance roundrobin
    option httpchk GET /health
    http-check expect status 200
    server aurora1 aurora1.local:8080 check
    server harmony1 harmony1.local:8081 check
    server aurora2 aurora2.local:8080 check backup
```

Database Replication

Primary-Replica Setup

Primary (postgresql.conf):

```
wal_level = replica
max_wal_senders = 5
wal_keep_size = 64
```

Replica:

```
pg_basebackup -h primary.local -D /var/lib/postgresql/data \
    -U replication -P -v -R -X stream
```

Redis Sentinel

```
# Sentinel configuration
sentinel monitor helixcode redis-master 6379 2
sentinel auth-pass helixcode your_redis_password
sentinel down-after-milliseconds helixcode 5000
sentinel parallel-syncs helixcode 1
sentinel failover-timeout helixcode 10000
```

Monitoring and Maintenance

Health Checks

```
#!/bin/bash
# health-check.sh

check_aurora() {
    curl -f http://localhost:8080/health || return 1
}

check_harmony() {
    curl -f http://localhost:8081/health || return 1
}

check_postgres() {
    pg_isready -h localhost -p 5432 -U helixcode || return 1
}

check_redis() {
    redis-cli -a "$REDIS_PASSWORD" PING | grep -q PONG || return
    1
}

# Run checks
check_aurora && echo "Aurora: OK" || echo "Aurora: FAIL"
check_harmony && echo "Harmony: OK" || echo "Harmony: FAIL"
check_postgres && echo "PostgreSQL: OK" || echo "PostgreSQL:
    FAIL"
check_redis && echo "Redis: OK" || echo "Redis: FAIL"
```

Backup Strategy

```
#!/bin/bash
# backup.sh

BACKUP_DIR="/backup/helixcode"
DATE=$(date +%Y%m%d_%H%M%S)

# Database backup
pg_dump helixcode | gzip > "$BACKUP_DIR/db_$DATE.sql.gz"

# Configuration backup
tar -czf "$BACKUP_DIR/config_$DATE.tar.gz" /etc/helixcode/
```

```
# Data backup
tar -czf "$BACKUP_DIR/data_${DATE}.tar.gz" /var/lib/helixcode/

# Cleanup old backups (keep 7 days)
find "$BACKUP_DIR" -name "*.gz" -mtime +7 -delete
```

Monitoring Integration

Prometheus Metrics

```
monitoring:
  prometheus:
    enabled: true
    endpoint: "/metrics"
    port: 9090
```

Grafana Dashboard

Import dashboard from scripts/monitoring/helixcode-dashboard.json

Security Hardening

Firewall Configuration

```
# Aurora OS
sudo firewall-cmd --permanent --add-port=8080/tcp
sudo firewall-cmd --reload

# Harmony OS
sudo firewall-cmd --permanent --add-port=8081/tcp
sudo firewall-cmd --permanent --add-port=8091/tcp
sudo firewall-cmd --permanent --add-port=8092/tcp
sudo firewall-cmd --reload
```

SELinux Configuration

```
# Allow network connections
sudo setsebool -P httpd_can_network_connect 1

# Allow database connections
sudo setsebool -P httpd_can_network_connect_db 1
```


File Permissions

```
# Binary
sudo chmod 755 /opt/helixcode/aurora-os
sudo chmod 755 /opt/helixcode/harmony-os
```

```
# Configuration
sudo chmod 640 /etc/helixcode/*.yaml
sudo chmod 600 /etc/helixcode/*.env
```

```
# Logs
sudo chmod 755 /var/log/helixcode
sudo chmod 644 /var/log/helixcode/*.log
```

```
# Data
sudo chmod 750 /var/lib/helixcode
```

Troubleshooting

Common Issues

Service Fails to Start

Symptoms: Service immediately exits after start

Diagnosis:

```
# Check logs
sudo journalctl -u helixcode-aurora -n 50
sudo journalctl -u helixcode-harmony -n 50

# Check configuration
helixcode-aurora --config /etc/helixcode/aurora-config.yaml --
    validate
helixcode-harmony --config /etc/helixcode/harmony-config.yaml --
    validate

# Check permissions
ls -la /opt/helixcode/
ls -la /etc/helixcode/
ls -la /var/log/helixcode/
```

Solutions: - Fix configuration syntax errors - Verify database connectivity - Check file permissions - Ensure environment variables are set

Database Connection Failed

Symptoms: “connection refused” or “authentication failed”

Diagnosis:

```
# Test connection
psql -h localhost -U helixcode -d helixcode

# Check PostgreSQL status
sudo systemctl status postgresql

# Check pg_hba.conf
sudo cat /etc/postgresql/*/main/pg_hba.conf
```

Solutions: - Verify database password in environment file - Check PostgreSQL is running - Update pg_hba.conf to allow connections - Restart PostgreSQL: `sudo systemctl restart postgresql`

Port Already in Use

Symptoms: “bind: address already in use”

Diagnosis:

```
# Check what's using the port
sudo lsof -i :8080
sudo ss -tlnp | grep 8080
```

Solutions: - Change port in configuration - Stop conflicting service - Kill process using the port (if safe)

High Memory Usage

Symptoms: System running out of memory

Diagnosis:

```
# Check memory usage
free -h
ps aux --sort=-%mem | head -10

# Check HelixCode memory
ps aux | grep -E "aurora-os|harmony-os"
```

Solutions:

```
# Reduce concurrent tasks
workers:
  max_concurrent_tasks: 5

# Enable resource limits
harmony:
  resource_manager:
    max_memory: "2GB"
```

Platform-Specific Issues

Aurora OS: Security Features Not Working

```
# Check Aurora OS version
cat /etc/aurora-release

# Verify security module
helixcode-aurora security diagnose

# Check logs
tail -f /var/log/helixcode/security.log
```

Harmony OS: NPU Not Detected

```
# Check NPU device
ls -la /dev/davinci*

# Verify drivers
lsmod | grep davinci

# Check Harmony OS version
cat /etc/harmonyos-release

# Test NPU
helixcode-harmony device npu-test
```

Maintenance Tasks

Regular Maintenance

Daily: - Check service status - Review error logs - Monitor disk space

Weekly: - Review performance metrics - Check backup integrity - Update system packages

Monthly: - Database maintenance (VACUUM, REINDEX) - Log rotation verification - Security updates

Maintenance Commands

Database maintenance

```
sudo -u postgres psql helixcode -c "VACUUM ANALYZE;"
```

```
sudo -u postgres psql helixcode -c "REINDEX DATABASE helixcode;"
```

Clear old logs

```
sudo find /var/log/helixcode/ -name "*.log.*" -mtime +30 -delete
```

Check disk usage

```
du -sh /var/lib/helixcode/*
```

```
du -sh /var/log/helixcode/*
```

Support and Resources

Documentation

- [Aurora OS Guide](#)
- [Harmony OS Guide](#)
- [Quick Start Guide](#)
- [Configuration Reference](#)

Scripts

- scripts/deploy-aurora-os.sh
- scripts/deploy-harmony-os.sh
- scripts/deploy-specialized-platforms.sh

Community

- GitHub Issues: <https://github.com/helixcode/helixcode/issues>
 - Documentation: <https://docs.helixcode.dev>
 - Forum: <https://forum.helixcode.dev>
-

Document Version: 1.0.0 **Last Updated:** 2025-11-07 **Platforms:** Aurora OS, Harmony OS